

VoxMind

Conversational AI Voice Assistant with Grounded RAG, Emotion Analysis and Secure Multi-User Architecture

A full-stack, multi-user system that transcribes speech, attributes speakers, reads emotional and language signals, and answers questions from a user's own documents with validated citations. Built as a portfolio project and tested as if it would be run for real.

How to read the numbers in this document. Every metric is copied from the repository's own documentation and labelled as an *automated test result*, an *evaluation benchmark result*, or a *live verification*. Where something is implemented but was not verified, or is documented as unavailable, the text says so. Sources are named next to each figure.

1. Project Overview

VoxMind is a conversational intelligence platform. A signed-in user can speak or type, upload audio and documents, and get back a transcript with speaker labels, per-turn emotion and language signals, and answers to questions that are grounded in the documents they uploaded, with the supporting evidence shown next to the answer.

The problem it addresses is the gap between a voice demo and a system you could actually rely on. A basic voice assistant transcribes and talks; it does not tell you where an answer came from, does not separate speakers, does not admit when the source material does not contain the answer, and rarely isolates one user's data from another's. VoxMind is designed around those gaps. It is aimed at settings where a spoken interaction or a set of documents needs to be understood and answered from, such as support conversations, and where an unsupported answer is worse than "I don't have that information."

The central technical idea is that **trust is enforced by the application, not requested from the model**. The LLM is asked to cite only the passages it was given, but its citations are checked against what retrieval actually returned, invalid ones are dropped before anything reaches the client, and the "grounded / partially grounded / ungrounded" label is computed by the application. Each answer then passes a guardrail layer before it is stored or spoken.

It differs from a basic chatbot or voice assistant in four ways: it is multimodal (speech, speaker turns, tone and text are all analysed); its answers are evidence-backed and refuse when evidence is missing; every heavy stage runs as an isolated, recoverable background job; and it is a multi-user system with ownership checks, rotating sessions, CSRF protection and rate limiting rather than a single-user demo.

2. The Problem

Each item below is a problem the codebase actually addresses.

Problem	How VoxMind addresses it
Conversational interaction across voice and text	One conversation workspace with a text composer, a voice control, and a message timeline. A spoken question and a typed question go through the same answer path (<code>RagService.generate_for_message</code>).
Speech-to-text from uploaded or recorded audio	ffmpeg normalisation to 16 kHz mono, then faster-whisper. Uploads are validated by content sniffing, not by the client's declared type.
Speaker attribution	pyannote.audio diarization, merged with the transcript by a deterministic alignment algorithm. When diarization is unavailable the pipeline still returns a transcript and reports the step as unavailable.
Emotion and context signals	A trained emotion classifier per speaker turn (one of several registered model versions), real sentiment and named-entity models, rule-based intent and topics, and a "semantic-vocal incongruence" signal. The incongruence signal is explicitly <i>not</i> a deception detector; this is enforced by a database <code>CHECK</code> constraint.
Retrieval and grounding	Hybrid retrieval (vector plus full-text, fused, then reranked) over the user's own documents.
Citations and evidence	Every answer carries citations; each is validated against the chunks that were actually retrieved. The UI shows the retrieved sources, their scores, and the cited excerpt.
Hallucination and unsupported claims	Invalid citations are dropped; <code>grounding_status</code> is computed by the app; when nothing relevant is retrieved the answer is marked <code>ungrounded</code> , and a guardrail layer can replace or block a response.
Conversation state	A bounded window of recent turns plus a rolling long-term summary, assembled into a token-budgeted context.
Secure multi-user isolation	Per-user ownership on every conversation-scoped resource, rotating and reuse-detecting refresh tokens, CSRF protection, and Redis-backed rate limiting.
Reliability of slow ML work	Heavy stages run on a Celery worker with retries, an idempotency guard, a poison-pill bound, orphan fencing, and scheduled reconciliation of abandoned runs.

3. Product Experience

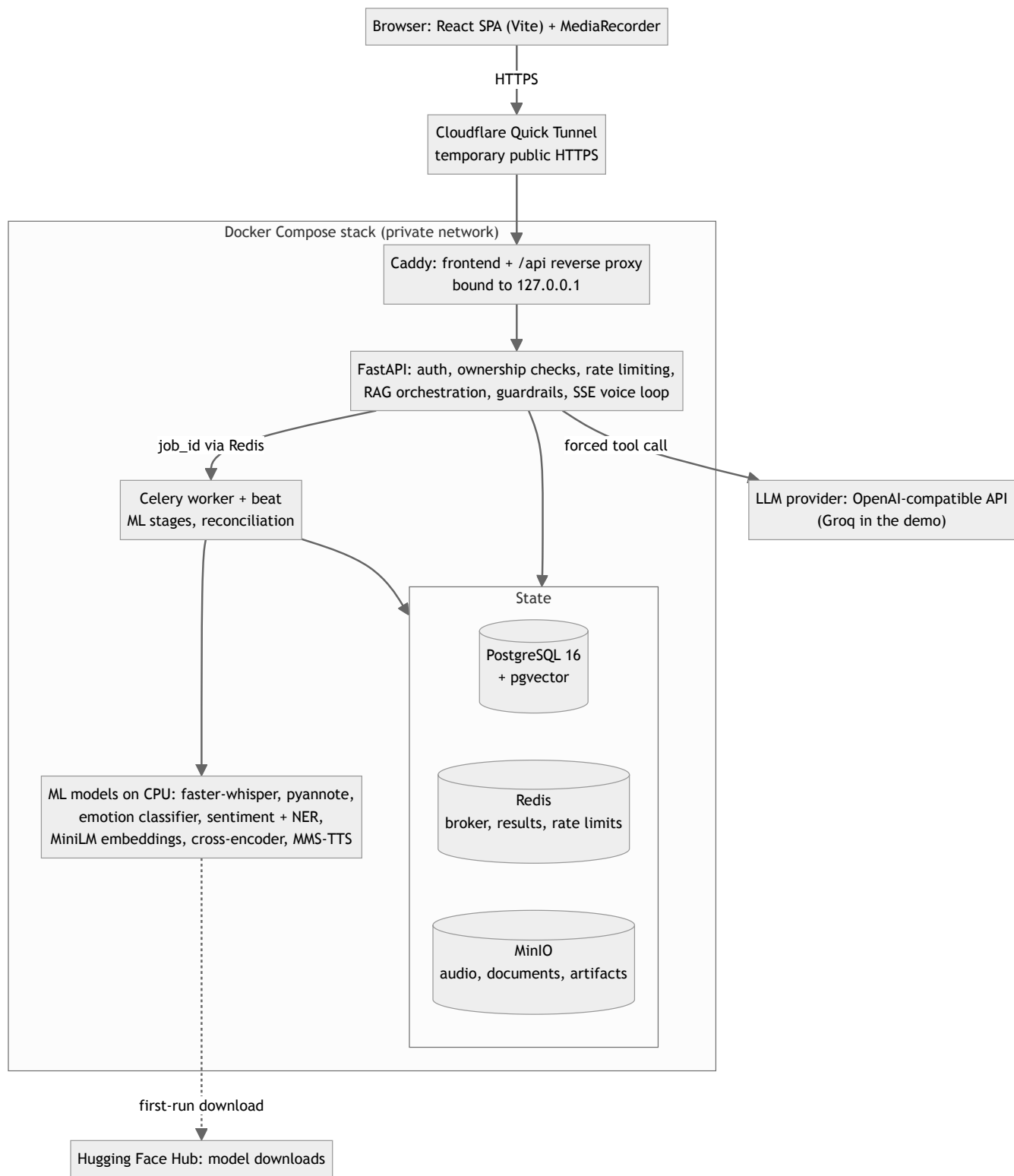
What a user does and what they see, in order:

- 1. Landing and account.** A landing page with *Get started* and *Sign in*. Registration and login use email and password.
- 2. Workspace.** After creating a conversation the user sees a central column (voice control, message timeline, text composer) and a right-hand column with four panels: *Audio & transcription*, *Speaker & emotion*, *Language understanding*, and *Knowledge*.
- 3. Add knowledge.** The user uploads a `.txt` , `.md` or `.pdf` document and presses *Ingest*. The panel shows the document with its chunk count once it has been parsed, chunked and embedded.
- 4. Ask a question.** The user asks in the Knowledge panel. The answer appears in the timeline. In the panel the user can see *Retrieved sources* (each chunk marked as reranked), the *Answer*, *Citations* with the quoted excerpt, and a grounding label such as "Grounded in retrieved sources" or "Not grounded in any retrieved source."
- 5. Ask something the documents do not contain.** VoxMind says it cannot find that information and marks the answer as not grounded, rather than inventing one.

6. **Upload audio.** After *Transcribe*, the transcript appears as speaker-labelled turns with timestamps (`speaker_0` , `speaker_1`).
7. **Analyse emotion.** *Analyze emotion* returns a label and a confidence per speaker turn, shown as "model output, not a verified fact."
8. **Speak to it.** The microphone control records a question. The interface moves through listening, transcribing, analysing, retrieving and speaking states, then plays the spoken reply. A *Timing breakdown* shows measured latency for speech-to-text, emotion and language analysis, retrieval, LLM generation and speech synthesis.
9. **History and insights.** A per-conversation *Insights* page shows an intelligence timeline (sentiment, intent, emotion, incongruence per message) with short observations. An *Analytics* dashboard aggregates the user's own measured data. An *Evaluation* dashboard shows the latest recorded evaluation runs.

Flow: text or voice input -> (audio preprocessing -> transcription -> speaker attribution) -> NLP and emotion -> memory + hybrid retrieval -> LLM -> citation validation and grounding -> guardrails -> stored answer -> (local text-to-speech) -> history .

4. System Architecture



Components

Component	Role
React frontend	Single-page app: auth pages, conversation workspace, voice control, analytics, insights and evaluation dashboards. The access token is held in memory only.

Component	Role
Caddy	Serves the built frontend and reverse-proxies <code>/api/*</code> to the API on one origin. In the deployment it binds to <code>127.0.0.1</code> only and rewrites <code>X-Forwarded-For</code> so the API's rate limiter cannot be spoofed.
FastAPI	REST API and the voice loop over Server-Sent Events. Owns authentication, ownership checks, rate limiting, RAG orchestration, and the guardrail layer.
PostgreSQL + pgvector	System of record: users, conversations, transcripts, predictions, chunks with <code>vector(384)</code> embeddings, retrieval traces, guardrail decisions, evaluation runs, and the authoritative status of every background job (<code>pipeline_runs</code>).
Redis	Celery broker and result backend, and the counters behind rate limiting.
Celery worker (+ beat)	Executes model-heavy stages on two queues (an ML queue and a cheap default queue). Beat schedules reconciliation of abandoned runs. Only a small <code>job_id</code> crosses the broker, never the payload.
MinIO	S3-compatible object storage for uploaded audio, documents and emotion-model artifacts, behind a storage abstraction that also has a local-filesystem backend.
ML models	Speech, diarization, emotion, NLP, embeddings, reranking and text-to-speech, all run on CPU.
LLM provider	An OpenAI-compatible client (used against Groq in the demo deployment) and an Anthropic provider, chosen by configuration. If no provider is configured, the answer step reports itself unavailable. The Anthropic provider is implemented and unit-tested but was not exercised against the live API.

5. AI / Voice Pipeline

Each stage below exists in the repository and is wired into the pipeline. Stages run through a common `PipelineStage` contract, so the same code runs in-process (default, for development and tests) or on the Celery worker.

Stage	What it does	Why it exists
Audio preprocessing	Decodes any accepted format (wav, mp3, m4a, ogg, webm) with ffmpeg via pipes into 16 kHz mono PCM, and validates the result. The original upload is never modified.	Gives every downstream model the same input format, and rejects undecodable files early with a clear error.
Speech-to-text	faster-whisper (Whisper via CTranslate2). The deployed configuration uses the <code>tiny</code> model, int8, on CPU.	Turns speech into text without any external service or credential.
Speaker diarization	pyannote.audio 3.1, gated on Hugging Face and enabled by a token.	Attributes each stretch of speech to a speaker. When unavailable, the transcript still succeeds and diarization is reported as unavailable.
Alignment	A deterministic algorithm merges transcript segments with speaker segments into speaker-attributed turns.	Produces the <code>speaker_0 / speaker_1</code> transcript, and is fully unit-testable because it involves no model.

Stage	What it does	Why it exists
Emotion analysis	Acoustic features (librosa) plus embeddings feed a trained classifier, per speaker turn. Several model generations are supported behind one interface: two Wav2Vec2-based models and an emotion2vec+ head. A model registry records which version produced each prediction.	Adds a vocal-tone signal alongside the words. If no trained model is registered, the step reports "unavailable" rather than inventing a label.
NLP	A RoBERTa sentiment model, a BERT named-entity model, rule-based intent, and keyword topics.	Provides language-side signals. The docs label which parts are neural and which are deterministic.
Incongruence	Compares each turn's sentiment with its predicted emotion.	An analytical divergence signal, explicitly not a lie detector.
Embeddings	<code>all-MiniLM-L6-v2</code> , loaded directly with Hugging Face Transformers, producing 384-dimension vectors stored in pgvector.	Enables semantic retrieval.
Retrieval + reranking	See section 6.	Finds and orders the evidence.
Grounded generation	A forced structured tool-call to the LLM, then application-side citation validation and grounding, then guardrails.	Keeps the answer tied to evidence and stops unsupported content reaching the user.
Text-to-speech	<code>facebook/mms-tts-eng</code> , run locally. An OpenAI TTS provider also exists but is credential-gated and was not used.	Lets the voice loop complete with no external speech service.
Streaming voice loop	The voice endpoint streams Server-Sent Events, one event as each real stage completes, ending in a <code>done</code> event with the transcript, answer, citations, grounding status, audio URL and stage timings. Cancelling the request records the turn as <code>interrupted</code> .	Gives the user genuine progress and lets them interrupt.

Two scope notes from the project's own documentation. First, the streaming is **stage-level**: token-by-token LLM streaming is not implemented, because the answer is a forced structured output that must be validated against retrieved chunks. Second, a thread already running a CPU-bound model call cannot be killed when a turn is interrupted; its result is discarded.

6. Retrieval-Augmented Generation

This is the part of the system with the most engineering behind it.

Ingestion. A document (`.txt`, `.md`, `.pdf`, size-limited) is stored through the storage abstraction and a `KnowledgeDocument` row is created. A background job then parses it (text decoded directly; PDFs through `pypdf`, so an image-only PDF honestly yields little text), and splits it with a deterministic, paragraph-aware chunker with configurable size and overlap. Each chunk is embedded and stored as a `vector(384)` column. Document status (`pending` / `processing` / `completed` / `failed`) and any error are recorded, never a fabricated success.

Retrieval. For each question the retrieval service:

1. **Vector search:** cosine-distance nearest neighbours in pgvector, backed by an `ivfflat` index (`lists=10`, sized for demo-scale data).

2. **Lexical search:** PostgreSQL full-text search (`to_tsvector` / `plainto_tsquery` , ranked with `ts_rank_cd` , GIN-indexed).
3. **Fusion:** deterministic Reciprocal Rank Fusion (`score = sum of 1/(k + rank)` , `k = 60`), rather than a hand-tuned weighting.
4. **Reranking:** a cross-encoder (`cross-encoder/ms-marco-MiniLM-L-6-v2`) scores each query and candidate pair jointly. If reranking is off or fails to load, results pass through with a null rerank score.
5. **Evidence selection:** the top `RETRIEVAL_TOP_K` chunks (default 5) are returned, and the full fusion trace (every candidate, its vector and lexical ranks, hybrid and rerank scores, and whether it was selected) is persisted, so a retrieval run can be inspected afterwards.

Context. The LLM never sees a raw database dump. A single assembler builds the context from the question, recent turns, the conversation summary, signals for the current turn and the retrieved chunks, and trims to a token budget in a fixed order (oldest turns, then chunk excerpts, then the summary), never the question or instructions.

Guardrails before generation. Retrieved chunks are scanned for prompt-injection patterns and any matching chunk is excluded from the model's context before it is assembled.

Citations and grounding. The model must reply through a single forced tool call. Every returned citation is checked against the chunk IDs that were actually retrieved, and invalid ones are dropped before persistence. Grounding status is computed from that check:

Status	Meaning
<code>grounded</code>	Every citation was valid and at least one was returned
<code>partially_ground</code>	Some citations were valid, some were dropped
<code>ungrounded</code>	No valid citations, or nothing was retrieved
<code>unavailable</code>	No LLM configured, or the call failed

Refusal. When nothing relevant is retrieved, the answer is marked `ungrounded` and the UI says so. After generation a guardrail layer validates structure and citations and runs a safety check, and can approve, modify or block the response. The raw model output is kept only in an audit field and is never returned to clients.

Verified retrieval metrics (evaluation benchmark result, run 2026-09-06; source: `docs/evaluation.md` , "Retrieval evaluation"). The benchmark runs the real ingestion, hybrid search, fusion and reranking code path over a small, hand-labelled collection.

Metric	Reported value
Queries evaluated (n_queries)	6
MRR	1.0000
Recall@1 / @3 / @5	1.0 / 1.0 / 1.0
Precision@1 / @3 / @5	1.0 / 0.33 / 0.20
nDCG@1 / @3 / @5	1.0 / 1.0 / 1.0

Notes that the source document itself makes, and that matter for reading this table: the collection is four short documents, five chunks and six queries, authored for the evaluation; each query has exactly one unambiguous relevant chunk on an unrelated topic. The documentation states that a perfect score is expected on a set this

small and "demonstrates the real pipeline and metric computation work correctly, not that production-scale retrieval is perfect." The evaluation defines no pass/fail thresholds; the values are reported at the cutoffs K = 1, 3 and 5. Precision@3 and @5 are low by construction, since each query has one relevant chunk.

7. Evaluation

VoxMind's evaluation framework treats every metric as being in one of four states: *implemented*, *executable*, *actually evaluated*, or *unavailable*. Results are stored insert-only in an `evaluation_runs` table (dataset version, model version, configuration, sample count, status, metrics, per-sample errors), and a dashboard shows the latest run per type or `never_run` if none exists.

Automated test results

Suite	Result	Source
Backend (pytest, real Postgres)	358 passed, 5 skipped, 0 failed (363 total; the 5 skips are credential or broker gated)	<code>docs/FINAL_PRODUCTION_FREEZE.md</code> , final freeze pass
ML / evaluation-logic tests	100 passed, 0 failed	same
Frontend (Vitest)	52 passed, 0 failed	same
Type checking and linting	<code>tsc -b clean; oxlint clean; ruff clean; mypy voxmind clean</code> (195 source files); <code>mypy ml/ clean</code>	same
GitHub Actions CI	Five runs across three pushed passes, all <code>conclusion: success</code>	same
Re-run during deployment work, CI-equivalent selection (excludes 21 real-model tests)	Backend 352 passed, 5 skipped; ML 100 passed; frontend 52 passed; <code>mypy voxmind</code> 196 source files clean	live run in this project (see note)

Note on the last row: it includes 15 credential-routing regression tests added after the freeze pass, and it excludes the real-model tests that the freeze figure includes, so the two backend numbers are not directly comparable. That later work is described in sections 8 and 10.

Evaluation benchmark results (run 2026-09-06; source: `docs/evaluation.md` unless stated)

Evaluation	Reported result	Context stated in the source
Speech-to-text (faster-whisper <code>tiny</code> , CPU)	n=5; avg_wer=0.0500; avg_cer=0.0363; avg_latency_ms=524.6	Small dataset. Only 1 of the 5 references is independent ground truth; the other 4 were written by the AI assistant that built the dataset and spoken by a synthetic voice.
Emotion, <code>ravdess-v1</code> (n=180, held-out speaker-independent RAVDESS split, 8 classes)	accuracy=0.6722; macro_f1=0.6899; weighted_f1=0.6779; balanced_accuracy=0.6823	Same protocol as the original training evaluation, reused unmodified.

Evaluation	Reported result	Context stated in the source
Emotion, <code>emotion2vec-tess-emodb-v1</code> , in-domain (n=92, in practice all EMO-DB)	Accuracy 0.9891; Macro F1 0.9867; Weighted F1 0.9890; Balanced accuracy 0.9833	The source notes TESS contributes no validation or test clips, that the <code>disgust</code> class has n=1, and that the near-perfect result reflects easy, studio-quality corpora. Source: <code>docs/emotion.md</code> .
Emotion, <code>emotion2vec-tess-emodb-v1</code> , zero-shot on RAVDESS (n=132, six-class mapping)	Accuracy 0.8182; Macro F1 0.8060; Weighted F1 0.8166; Balanced accuracy 0.8194	Never trained on RAVDESS; the source states this is not a like-for-like comparison with the 8-class model. Source: <code>docs/emotion.md</code> .
Retrieval	See section 6	Small, hand-labelled set.
Grounding	<code>n_generations=3</code> ; <code>grounding_status_distribution={grounded: 1, ungrounded: 2}</code> ; <code>citation_validity_rate=1.0</code> (1/1 citations); <code>unsupported_answer_rate=UNAVAILABLE</code>	Answer-content support needs an entailment model or human judgment, which the repository does not have, so it is reported as unavailable rather than approximated.
System (development environment)	<code>voice_turns</code> n=12, <code>completion_rate=0.125</code> (1 completed / 8 resolved), total latency p50=p95=10945 ms (n=1); <code>llm_generations</code> n_real_attempts=3, <code>failure_rate=0.0</code>	The source describes these as a lightly used development environment, "not a production SLA claim."

Licensing note from the repository: the emotion2vec+ head was trained partly on the TESS corpus (CC BY-NC-ND 4.0, non-commercial), and the README states that the resulting checkpoint inherits that restriction.

Live verification (a real stack, exercised over HTTP and in a real browser; source: `docs/FREE_DEPLOYMENT.md`, "What was verified")

- `scripts/smoke_test.py` against the running stack: 48 of 48 checks passed through a public Cloudflare Quick Tunnel over HTTPS; 47 passed locally, plus one intentionally skipped (the rate-limit step).
- A scripted Chromium journey through the public tunnel: 14 of 14 steps (account, conversation, text, document ingest, grounded answer with a retrieved source, audio upload and transcription, speaker-labelled transcript, emotion, voice turn, synthesized audio response).
- Security behaviours covered by the smoke test: refresh rotation and reuse rejection, CSRF rejection, cross-user isolation returning 404, non-audio content disguised as `.wav` rejected with 415, and rate limiting returning 429.
- **Security and isolation testing** is also part of the automated suite. Examples of named tests: cannot access another user's conversation, cannot ask in another user's conversation, cannot upload to another user's conversation, cannot view another user's insights, emotion job and processing job are ownership-protected, refresh without a CSRF header is rejected, reuse of a rotated refresh token revokes the token family, and the internal job table is never reachable over HTTP.

8. Security and Multi-User Isolation

Area	What is implemented
Authentication	bcrypt password hashing (never logged or returned). Short-lived JWT access tokens (15 minutes by default) held only in memory on the client. Rotating, reuse-detecting refresh tokens kept in an <code>HttpOnly</code> cookie scoped to the auth endpoints. Reusing an already-rotated token is treated as a compromise signal and revokes the session family, with a short grace window for legitimate concurrent refreshes.
CSRF	A separate, readable CSRF cookie is checked against a request header on refresh and logout. Browser QA found a session-loss bug caused by the CSRF cookie's path being too narrow; it was fixed by giving the CSRF cookie an app-wide path while keeping the refresh cookie narrow, with a regression test.
Authorization and isolation	Every conversation-scoped resource (messages, audio, transcripts, jobs, documents, memory, insights) is checked against the owning user. A resource owned by someone else returns 404, so its existence is not revealed. Storage backends also check ownership before any read or write.
Rate limiting	Redis-backed fixed-window limiter with four categories: auth (10 per window, keyed by IP), expensive ML/LLM work (15), document ingestion (20) and everything else (120), with <code>429</code> and <code>Retry-After</code> . The window is 60 seconds. The documented trade-off is up to about twice the budget across a window boundary.
Proxy trust	In the deployment, Caddy replaces <code>X-Forwarded-For</code> with Cloudflare's connecting-IP header (or the real TCP peer) instead of appending to it, because the limiter trusts the first entry. Forged headers were tested and did not evade the limiter.
Input validation	Audio type is determined by magic bytes, not the client's header; size and duration limits; safe filenames; document type and size limits; structured request schemas.
Secrets	Environment files are git-ignored and excluded from Docker build contexts; images are built without secrets. Each credential is used for exactly one provider: the LLM key is only sent to the configured LLM endpoint, and an optional, separate key is used only for OpenAI moderation. All OpenAI-compatible clients are built through one function that always pins an explicit base URL. A regression test suite captures outgoing requests and asserts each fake key reaches only its own host.
Broker safety	Celery serialisation is pinned to JSON, never pickle; only a job ID crosses the broker; the internal job-status table is structurally prevented from being exposed over HTTP.
LLM-specific defences	Prompt-injection scanning of retrieved content, application-side citation validation, guardrail decisions (approved, modified, blocked).
Audit trail	Each guardrail evaluation is persisted as a queryable row (decision, reasons, filtered chunks, safety categories and method, original and final answer). Evaluation runs are insert-only. Application logs are structured and can be JSON. There is no separate security-event audit log.

What the project does not claim. Its security document states there is no WAF, no intrusion detection, no penetration test, no third-party audit, and no secrets-manager integration. The moderation check in the demo deployment is the built-in keyword fallback, which is weaker than a hosted classifier and is documented as such.

9. Engineering Challenges and Solutions

1. A reranker that returned NaN on one CPU backend - Challenge: Every reranker score was `NaN`, which failed when the retrieval trace was saved as JSON. - **Why it mattered:** Reranking is central to answer quality, and the failure was silent until persistence. - **What changed:** Two causes were traced separately. The tokenizer

was being called with a list of pairs instead of two parallel lists, and, separately, the encoder produced NaN in float32 on this machine's CPU backend, reproduced with valid input. The tokenizer call was corrected and the model is loaded in float64. (ADR 0007) - **Result:** Reranker scores are computed, persisted, and shown in the UI. The same class of issue was later found and handled in a second model.

2. Twelve call sites that never waited for background jobs - Challenge: Six services dispatched pipeline stages and checked status exactly once. That was silently correct with the in-process runner and wrong with Celery. - **Why it mattered:** The default configuration and all CI runs used the in-process runner, so the bug was invisible to the test suite. - **What changed:** A bounded polling helper was added to the task-runner contract, and all twelve call sites now use it. (ADR 0011) - **Result:** Real Celery execution works through the same service code as the in-process path.

3. Recovering work abandoned by a lost worker - Challenge: If a worker died and the broker message was also gone, a job row stayed running forever. A late, orphaned worker could also overwrite a decision made after recovery. - **Why it mattered:** Slow ML jobs make crashes and restarts realistic. - **What changed:** An idempotent reconciliation function, scheduled through Celery Beat, resets or fails stale runs. Terminal writes use a database compare-and-swap on the row's own status and started_at, so a superseded attempt's write is rejected. A delivery-count cap bounds poison-pill redelivery. (ADR 0017; freeze document) - **Result:** A test drives a real mid-task write and confirms a stale worker's completion is rejected. A hard worker kill was verified to recover.

4. A guardrail that guarded the wrong field - Challenge: The stored answer held the model's raw output, while the API and speech layers read that field. A blocked answer's safe replacement would never have reached the client; the raw text would have. - **Why it mattered:** It defeated the purpose of the guardrail layer. - **What changed:** The guardrail now overwrites answer with the approved final text, and the raw output moves to a separate audit-only column that no client path reads. - **Result:** Integration tests confirm the raw unsafe text is absent from every client-readable field.

5. Building the container image on Python 3.11 - Challenge: A fresh dependency resolve failed on a real numpy conflict between the diarization and emotion libraries, and the default PyTorch wheel pulled in CUDA libraries that crash on a CPU-only container. - **Why it mattered:** The image would not build, or would fail at import. - **What changed:** Installation uses a frozen constraints file with dependency resolution disabled, and PyTorch is installed separately from its CPU-only wheel index. A soundfile ARM64 wheel gap was also fixed. (ADR 0016) - **Result:** The API and worker images build natively for linux/arm64 and run the full stack.

6. Fitting the ML stack into a 4 GB Docker VM - Challenge: Loading the emotion2vec+ checkpoint peaked at 2.76 GB in an otherwise idle process. Inside the worker, the process was OOM-killed every time in a 3.8 GiB VM, including with one task per process. - **Why it mattered:** It made the default emotion model unusable on the target machine. - **What changed:** The active emotion model is selectable at deployment time. The deployed configuration uses the lighter ravdess-v1 model, which was verified end to end. Worker process recycling was measured and set back to its default after the alternative proved slower. (docs/FREE_DEPLOYMENT.md) - **Result:** The full pipeline runs on the 8 GB laptop it was built on. emotion2vec+ as the active model was not verified and needs more memory.

7. A credential that could go to the wrong provider - Challenge: An OpenAI SDK client created without an explicit base URL follows the process environment, so where a key was sent depended on how the process was launched. In one launch mode the moderation client would send the LLM key to OpenAI's endpoint. - **Why it mattered:** A secret for one provider must never reach another. - **What changed:** All clients are built through one function that always sets a base URL. Moderation got its own optional key. Regression tests capture outgoing requests with a hostile ambient environment and static tests forbid direct client construction elsewhere. - **Result:** Keys are tested to reach only their own hosts; the tests fail on the old code.

8. Cancelling a streaming voice turn - Challenge: The first cancellation approach polled for client disconnect between stages; live testing showed that check never got to run. - **Why it mattered:** Interrupted turns need to be recorded reliably. - **What changed:** A handler catches the cancellation and performs a shielded database write so the cleanup is not itself cancelled. (ADR 0009) - **Result:** Interrupted turns are marked `interrupted`. The documentation also states the residual limitation: an already-running CPU-bound thread cannot be killed.

10. Deployment

The final deployment runs the whole stack on one machine with Docker Compose and exposes it temporarily through a Cloudflare Quick Tunnel. It costs nothing to run and needs no account or payment method.

- **Compose stack.** Six long-lived services on a private network: PostgreSQL with pgvector, Redis, MinIO, the API, one Celery worker with Beat embedded, and a proxy image that builds the React app and serves it through Caddy. A one-shot job creates the storage bucket. Only Caddy publishes a host port, and it is bound to `127.0.0.1`. Postgres, Redis and MinIO (including its console) are not published.
- **Persistence.** Named Docker volumes hold the database, object storage and the downloaded model cache. A warm-up step pre-downloads every model so the first request does not hit a task timeout.
- **Scripts.** `start-prod.sh` builds, starts, waits for health, applies migrations, seeds the emotion model registry and warms models. Separate scripts start and stop the tunnel and report status. All are idempotent.
- **Secrets and configuration.** One git-ignored environment file holds generated secrets (database password, signing key, storage keys) and optional external credentials: a Groq key for LLM answers and a Hugging Face token for the gated diarization model. Templates in the repository contain placeholders only. No key is placed in the frontend bundle or any image layer, which was verified by scanning the build, all image layers, logs and HTTP responses.
- **Temporary public access.** `cloudflared` runs a Quick Tunnel that forwards to the local Caddy port. **The public URL is temporary: it is generated when the tunnel starts, changes every time it is restarted, and stops working when the tunnel or the machine stops.** This is why no public URL is listed in this document. VoxMind is a demo hosted from a laptop, not an always-on service.
- **Known Quick Tunnel limits** (from Cloudflare's documentation): a cap of 200 concurrent requests, an official "testing and development only" status, and no guarantee for Server-Sent Events. Streaming worked in the recorded runs.
- **Runtime limits observed:** the emotion model limitation above; a voice turn takes roughly one to three minutes on this CPU-only laptop depending on load.

11. Demo

[Watch the VoxMind Demo Video]

The demo is a roughly 100-second edited screen recording of the live application. It shows, in order: creating an account; uploading a short fictional support guide and ingesting it; asking a question and scrolling the Knowledge panel through the retrieved sources, the citation excerpt and the "Grounded in retrieved sources" label; asking a question the document cannot answer and getting an honest "not grounded" refusal; uploading a two-speaker call and receiving a speaker-labelled transcript; running emotion analysis on a short clip; a voice turn with a spoken reply and a per-stage timing breakdown; and the Insights and Analytics pages.

Things the demo is transparent about: - **All inputs are synthetic.** The two-speaker call, the emotion clip and the spoken question were generated locally with text-to-speech voices. The support guide is for a fictional product. The account is a throwaway demo account. None of it is real user data. - **The microphone input is a prerecorded file** fed through the browser's fake capture device. It is not a live microphone. - **Long waits are**

sped up, and each sped-up segment carries an on-screen badge showing the real elapsed time. - **The emotion result comes with a caveat.** The classifier was trained on acted human speech, and synthetic voices are out of its domain. Of five synthetic clips tested, one was classified as intended; the demo uses that one, and it is labelled "model output, not a verified fact." The result is illustrative, not evidence of accuracy. - **Audio only.** VoxMind accepts audio formats (wav, mp3, m4a, ogg, webm). It does not process video.

12. Technical Stack

Category	Technology	Purpose
Frontend	React 19, Vite, TypeScript	Single-page application
Frontend	Tailwind CSS, framer-motion, Recharts	Styling, motion, dashboard charts
Frontend	TanStack Query, React Router, Zustand	Server state, routing, in-memory auth state
Frontend testing	Vitest, Testing Library, axe-core (jest-axe), oxlint	Unit and component tests, accessibility checks, linting
Backend	Python 3.11, FastAPI, Pydantic	Async REST API, SSE streaming, validation
Data	SQLAlchemy (async), Alembic, PostgreSQL 16	ORM, 9 migrations, relational store
Vector search	pgvector (<code>vector(384)</code> , ivfflat), PostgreSQL full-text search	Hybrid retrieval
Background jobs	Celery, Redis, Celery Beat	Queues, retries, scheduled reconciliation
Object storage	MinIO (S3 API) via boto3, local-filesystem backend	Audio, documents, model artifacts
Speech	ffmpeg, faster-whisper, pyannote.audio 3.1, soundfile	Preprocessing, transcription, diarization
Emotion	librosa, PyTorch, Wav2Vec2 embeddings, emotion2vec+ (FunASR), scikit-learn (metrics), MLflow (optional)	Features, models, evaluation
NLP and retrieval models	Hugging Face Transformers: RoBERTa sentiment, BERT NER, MiniLM embeddings, MS MARCO cross-encoder	Language signals, embeddings, reranking
Text-to-speech	facebook/mms-tts-eng (VITS), local	Spoken replies
LLM	OpenAI-compatible client (Groq in the demo deployment); Anthropic provider (implemented, not exercised live)	Structured, tool-call answer generation
Security	bcrypt, JWT, CSRF double-submit token, Redis rate limiter	Authentication and abuse protection
Infrastructure	Docker Compose, Caddy, cloudflared Quick Tunnel	Deployment and temporary public access
Quality	pytest, pytest-asyncio, ruff, mypy, GitHub Actions	Tests, static analysis, CI
Verification tooling	Playwright (Chromium), ffmpeg	Browser walkthroughs, demo recording

13. Key Engineering Decisions

1. **A stage contract with two runners.** Every pipeline stage implements one interface and runs either in-process or on Celery. This kept development and tests fast, and let Celery be added without rewriting services. The polling bug in section 9 is the cost of that seam being under-tested, and it was fixed at the contract level.
2. **pgvector inside Postgres rather than a separate vector database.** Embeddings sit next to the relational data, so ownership filtering, retrieval traces and joins use one system. The `ivfflat` index is sized for demo-scale data and is documented as such.
3. **Hybrid retrieval with reranking.** Vector search catches meaning, full-text search catches exact terms, Reciprocal Rank Fusion combines them without hand-tuned weights, and a cross-encoder reorders the shortlist by scoring query and passage together.
4. **Grounding enforced in application code.** The model is instructed to cite only what it was given, but its citations are validated against the retrieved set, invalid ones are dropped, and the status label is computed by the application. A guardrail layer sits between the model and everything downstream.
5. **Honest degradation instead of fake output.** If there is no LLM key, no trained emotion model, or no diarization token, the affected step reports `unavailable` and the rest of the pipeline continues. Nothing is filled in with a placeholder.
6. **Secrets separated per provider.** One credential per provider, one client-construction function that pins its host, an optional separate moderation key, and tests that assert routing.
7. **Resource constraints drove model selection.** The emotion model is chosen at deployment time based on measured memory, and the deployed configuration uses the model that fits.
8. **Same-origin frontend and API.** Caddy serves both under one origin, so the frontend uses relative URLs, cookies are host-only with `SameSite=Lax`, no cross-origin configuration is needed, and no hostname is baked into the build. That last point matters because the tunnel hostname changes on every run.
9. **A reconciliation loop over more infrastructure.** Abandoned jobs are recovered by an idempotent function that first ran as a command and was later scheduled with Celery Beat, a mode of a dependency already in use, rather than by introducing a new scheduler service.
10. **Evaluation states that refuse to blur.** A metric is implemented, executable, evaluated, or unavailable, and the framework reports "unavailable" for unsupported-answer rate rather than approximating it.

14. Results

Verified results, each with its type. These are not ranked.

Result	Value	Type
Backend tests at final freeze	358 passed, 5 skipped, 0 failed	Automated test result
ML tests	100 passed	Automated test result
Frontend tests	52 passed	Automated test result
GitHub Actions CI	Five runs, all successful	Automated test result
Static analysis	<code>mypy</code> <code>voxmind</code> clean (195 files at freeze), <code>mypy</code> <code>ml/</code> clean, <code>ruff</code> clean, <code>tsc</code> clean, <code>oxlint</code> clean	Automated test result

Result	Value	Type
Retrieval benchmark	MRR 1.0000; recall@1/3/5 = 1.0/1.0/1.0; nDCG@1/3/5 = 1.0/1.0/1.0 (n_queries=6, five chunks)	Evaluation benchmark result
Grounding	citation_validity_rate 1.0 (1/1 citations); unsupported_answer_rate UNAVAILABLE	Evaluation benchmark result
Speech-to-text (faster-whisper tiny)	avg_wer 0.0500, avg_cer 0.0363, avg_latency_ms 524.6 (n=5)	Evaluation benchmark result
Emotion, ravedess-v1, held-out speakers	accuracy 0.6722, macro_f1 0.6899 (n=180, 8 classes)	Evaluation benchmark result
Emotion, emotion2vec-tess-emodb-v1, zero-shot on RAVDESS	accuracy 0.8182, macro F1 0.8060 (n=132, six-class mapping)	Evaluation benchmark result
Public smoke test through the tunnel	48 of 48 checks	Live verification
Browser journey through the tunnel	14 of 14 steps	Live verification
Accessibility	An axe-core audit across 8 routes went from 19 violations to 0; documented as a targeted audit, not a formal WCAG certification	Live verification

15. What I Learned

- **An AI feature is only as trustworthy as its checks.** Asking a model to cite its sources did not make answers grounded. The trust came from validating citations against what retrieval returned and computing the grounding label in code. That changed how I think about where guarantees should live.
- **Evaluation has to be honest about its limits.** Building the evaluation framework meant deciding what counts as ground truth. The speech-to-text set has one independent reference out of five, the retrieval set is five chunks, and unsupported-answer rate could not be measured at all. Reporting those limits next to the numbers was more useful than a clean table.
- **Bugs hide behind defaults.** The polling bug was invisible because every test used the in-process runner. The credential-routing issue depended on whether an environment variable was exported. I now look for the configuration where a default masks a difference.
- **Debugging a numerical failure means bisecting.** The reranker NaN needed input checks, layer-by-layer inspection and a precision comparison, not a guess.
- **Asynchronous work needs a recovery story, not just a queue.** Retries and idempotency covered the common crash, but the interesting cases were the ones where the message was gone or a stale worker woke up late. Fencing writes with a compare-and-swap was the part that made recovery safe.
- **Measure before choosing a model.** I assumed the best emotion model would fit. Measuring showed a 2.76 GB peak against a 3.8 GiB VM, and that per-task worker recycling made things slower, so the deployment uses the model that fits and says so.
- **A working demo is not the same as a tested system.** Browser QA found a real session-loss bug that unit tests had not, and the smoke test and Playwright journeys found problems the unit suites could not see. Both now exist as repeatable scripts.
- **Security is mostly small, testable invariants.** Ownership returns 404, secrets go to one host each, a forged header does not evade the limiter. Each became a test that fails when the invariant breaks.

16. Future Improvements

All of the following are **future work** and are not implemented.

- **Run the stronger emotion model by default** on a machine with more memory, and verify it end to end. It was measured to load in isolation but was not verified as the active model in the deployment.
- **Reduce voice-turn latency.** The pipeline is CPU-only and a turn currently takes minutes on a loaded laptop. Keeping models resident, or moving to hardware that can run them faster, would help.
- **Token-level streaming** of answers. It is out of scope today because the answer is a forced structured output that must be validated; it would need a design that validates citations incrementally or after streaming.
- **Broader evaluation data.** A larger, independently labelled retrieval collection, a human-transcribed speech corpus, and an entailment or human-judgment method to measure unsupported-answer rate.
- **A stable hosted deployment** with a fixed hostname, in place of a temporary tunnel.
- **A live-microphone verification** by a human speaker, which the automated runs could not perform.
- **Moderation through a hosted classifier** instead of the keyword fallback used in the demo deployment.

17. Portfolio Summary

A) Two-sentence portfolio description

VoxMind is a multi-user voice and document assistant that transcribes speech, separates speakers, reads emotion and language signals, and answers questions from a user's own documents with citations checked against the retrieved evidence. It runs as a Docker Compose stack of FastAPI, Celery, PostgreSQL with pgvector, Redis, MinIO and local speech, emotion, retrieval and text-to-speech models, and is covered by backend, ML and frontend test suites and a live end-to-end smoke test.

B) One-line resume bullet

Built VoxMind, a multi-user voice and RAG assistant (FastAPI, React, Celery, PostgreSQL/pgvector) with application-validated citations and grounding, verified by 358 backend, 100 ML and 52 frontend tests at final freeze.

C) Three resume bullets

- Designed a hybrid retrieval pipeline (pgvector cosine search, PostgreSQL full-text search, Reciprocal Rank Fusion, cross-encoder reranking) with application-side citation validation and computed grounding status, so unsupported answers are marked ungrounded instead of shown as fact; scored MRR 1.0000 and recall@1/3/5 of 1.0 on a small hand-labelled benchmark (6 queries, 5 chunks).
- Made slow ML work reliable with Celery and Redis: idempotent stages, a poison-pill bound, compare-and-swap fencing against stale workers, and scheduled reconciliation of abandoned runs; fixed 12 call sites that never waited for job completion, a defect that only appeared with a real broker.
- Secured a multi-user API with rotating, reuse-detecting refresh tokens, CSRF protection, per-user ownership checks that return 404 across users, and Redis rate limiting; verified end to end with a 48-check smoke test through a public tunnel and a 14-step browser journey, and added regression tests asserting each API key is sent only to its own provider.

D) Keywords

Retrieval-Augmented Generation (RAG), pgvector, FastAPI, Celery, speech-to-text and speaker diarization, multi-user security, LLM guardrails and grounding, Docker Compose

